Flight Sequencer

Dual-Core RTOS Firmware Design

Technical Report — Architecture, Synchronization, Timing Analysis and Reference Implementation

Embedded Firmware Design Assessment
Launch-Vehicle Flight Sequencer on a Multi-Core MCU

Author: GSBiradar
Repository: github.com/GSBiradar/Skyroot-Dual-Core_RTOS_Flight_Sequencer
Source package: mission_computer/ (include, src, platform, config, scripts, docs)
2026-10-12

Contents

Contents 2

1. Executive Summary 4

2. Introduction 4

2.1 Purpose . . . 4

2.2 Scope . . . 4

2.3 Definitions and Abbreviations . . . 4

3. Requirements and Assumptions 5

3.1 Functional Requirements . . . 5

3.2 Non-Functional (Timing) Requirements . . . 5

3.3 Assumptions . . . 5

4. System Architecture 5

4.1 Multi-Core Strategy . . . 6

4.2 Low-Level Architecture . . . 6

4.3 Tasks, Timers and Interrupts . . . 7

4.4 Data Flow and IPC Map . . . 7

4.5 Hardware Interface Strategy: Interrupts versus Polling . . . 8

5. Timing and Determinism Analysis 9

5.1 Interrupt Load . . . 9

5.2 Worst-Case Response Times . . . 9

5.3 CPU Utilisation . . . 10

5.4 Data Age and Staleness . . . 11

5.5 Overrun and Missed-Release Detection . . . 11

6. Synchronization and Concurrency 12

6.1 Shared Resources . . . 12

6.2 The Seqlock Mailbox Protocol . . . 12

6.3 Correctness Arguments . . . 12

7. Implementation 13

7.1 File Layout . . . 13

7.2 The Board-Support Contract . . . 13

7.3 Sensor Ingest (T1) . . . 13

7.4 Flight Sequencer (T2) . . . 14

7.5 Slow Computation (T4) . . . 14

7.6 Supervisor and Watchdog (T3) . . . 14

8. Building and Running 14

8.1 Windows (MinGW-w64 gcc) . . . 14

8.2 Linux / WSL 15
8.3 Make and CMake 15
9. Validation **15**
9.1 Test Plan 15
9.2 Observed Results 15
10. Improvements over the Base Pseudocode **15**
11. Evaluation Traceability **16**
12. Porting and Integration **16**
13. Conclusion **17**
Appendix A. Constants and Frame Format **18**
Appendix B. Board-Support API Reference **18**
Appendix C. Repository Layout **18**
Appendix D. Glossary **19**

1. Executive Summary

This report documents a complete design and a working reference implementation of a Flight Sequencer for a launch-vehicle computer. The system runs on a dual-core 32-bit MCU under a preemptive, fixed-priority RTOS. The central engineering decision is an asymmetric core split: one core is reserved exclusively for the hard real-time Flight Sequencer, while the other core absorbs all asynchronous I/O, supervision and slow background computation. This isolates the 10 ms control loop from every source of jitter that can be removed by partitioning.

All cross-core and cross-rate data is exchanged through single-writer seqlock latest-value mailboxes and hardware-timer-driven task notifications. There are no mutexes, queues or kernel calls on the hard real-time path, so the highest-priority consumer never waits on a lower-priority producer. Sensor reception uses DMA plus an idle-line interrupt with validation deferred to a high-priority ingest task; transmission uses DMA with polled completion because the timing is controlled by the flight software.

The package is executable: the same application code that targets FreeRTOS-SMP compiles and runs on Windows and Linux through a host board-support port that simulates both cores, four RS422 ports, the hardware timers, the watchdog and an injectable fault model. A three second run reproduces roughly 300 sequencer cycles, 600 sensor-1 frames and 300 sensor-2 frames, with the safety paths (CRC rejection, stale-data hold, non-blocking transmit and watchdog gating) all exercised and observed in the health report.

2. Introduction

2.1 Purpose

The purpose of this document is to present, in a single place, the requirements, architecture, synchronization strategy, timing analysis, implementation, build/run instructions, validation evidence and porting guidance for the Flight Sequencer. It is written to be read from the top down: the early sections give the context and the requirements, the middle sections give the design and the code, and the appendices give the reference detail.

2.2 Scope

In scope: task and interrupt design, inter-process communication (IPC), synchronization and concurrency correctness, interrupt-vs-polling strategy, real-time determinism and response-time reasoning, and a runnable C reference implementation with a host simulation. Out of scope: mission-specific flight dynamics. The guidance algorithms are deterministic, bounded placeholders because the navigation payload semantics and a mission interface-control document are not part of the assessment. The architecture preserves the timing contract that a real algorithm would have to satisfy.

2.3 Definitions and Abbreviations

Term	Meaning
ISR	Interrupt Service Routine
DMA	Direct Memory Access — hardware moves bytes without the CPU
IPC	Inter-Process Communication
WCET	Worst-Case Execution Time
Seqlock	Sequence-lock: a lock-free, wait-free-writer data exchange
Mailbox	A fixed-size single-writer shared buffer with latest-value semantics
FDIR	Fault Detection, Isolation and Recovery
Watchdog	Hardware timer that resets the system unless periodically kicked
Frame	One RS422 message: 2 B sync + 1 B seq + 1 B len + payload + 2 B CRC16

3. Requirements and Assumptions

3.1 Functional Requirements

- Acquire Sensor 1 (32 B) every 5 ms and Sensor 2 (54 B) every 10 ms over RS422.
- Run a Flight Sequencer control loop every 10 ms that consumes the latest sensor data and a 16-byte feedback value, and produces a 20-byte actuator command and a 16-byte output for the slow task.
- Run a slow computation every 200 ms that consumes navigation and sequencer output and produces 16-byte feedback applied back into the loop.
- Run a 10 ms supervisor that monitors health and emits telemetry.
- Never block the control loop waiting for data; on stale or invalid input, hold the last good value, raise a flag and degrade safely.
- Feed a watchdog only when the monitored health is valid.

3.2 Non-Functional (Timing) Requirements

Activity	Period / trigger	Deadline	WCET budget
Sensor 1 ingest	5 ms (event)	before next 5 ms frame	40 us/frame
Sensor 2 ingest	10 ms (event)	before next 10 ms frame	40 us/frame
Flight Sequencer	10 ms	10 ms	3 ms
Supervisor	10 ms (phase +6 ms)	10 ms	200 us
Slow computation	200 ms	200 ms	40 ms

3.3 Assumptions

The brief leaves hardware details open; the following assumptions are made explicit and can be revised without changing the structure of the design.

#	Assumption
A1	Dual-core 32-bit MCU at 200 MHz or faster, shared on-chip SRAM, hardware atomics (LDREX/STREX), a shared free-running 1 microsecond timebase readable by both cores, per-interrupt core routing, and a hardware watchdog.
A2	Shared RAM is mapped non-cacheable through the MPU (or is hardware-coherent). Every shared object is 64-byte aligned to avoid false sharing.
A3	The RTOS is SMP, preemptive and fixed-priority (higher number = higher priority) with hard per-task core affinity, a 1 kHz tick, and static allocation only (no heap after init).
A4	RS422 ports are UARTs at 1 Mbaud, 8N1 (10 microseconds per byte). Each has its own DMA channel (circular RX or one-shot TX) and idle-line detection.
A5	Frame format = 2 B sync + 1 B sequence + 1 B length + payload + 2 B CRC16 (6 B overhead). Wire times: Sensor 1 (38 B) 0.38 ms, Sensor 2 (60 B) 0.60 ms, actuator (26 B) 0.26 ms, telemetry (64 B) 0.64 ms.
A6	Sensors push frames autonomously at exact periods; their phase relative to the MCU is unknown. The design works for any phase.
A7	WCET budgets: ISR 5 us, sensor ingest 40 us per frame, Flight Sequencer 3 ms, Supervisor 200 us, slow task 40 ms.
A8	Failure policy: cyclic tasks never block waiting for data. Stale or invalid input means hold the last good value, raise a flag, and enter degraded mode.
A9	The 200 ms feedback is applied only at a fixed frame boundary (every 20th frame), so behaviour is repeatable regardless of when the slow task finishes.

4. System Architecture

4.1 Multi-Core Strategy

The design partitions work so that the hard real-time control loop has a dedicated core.

- Core 1 is the deterministic compute core. It runs only the Flight Sequencer task and its 3 microsecond timer ISR. No UART interrupt, no logging and no lower-priority task can run there, so jitter comes only from the timer ISR and shared-bus contention.
- Core 0 is the I/O and housekeeping core. It absorbs all asynchronous events (sensor frame interrupts), ingest, supervision and the slow computation. Its load is about 23 percent, and the slow task is lowest priority, so everything else preempts it and it can never delay ingestion.
- Contention is minimised because every shared object has exactly one writer and a wait-free write path. The cores communicate only through four small mailboxes. There are no cross-core mutexes, queues or kernel calls on the critical path.
- Why the slow task is on Core 0: placing a 40 ms task on Core 1 would let it interfere with the hard 10 ms task and would put the cross-core boundary in the wrong place. Core 1 keeps spare capacity for growth.

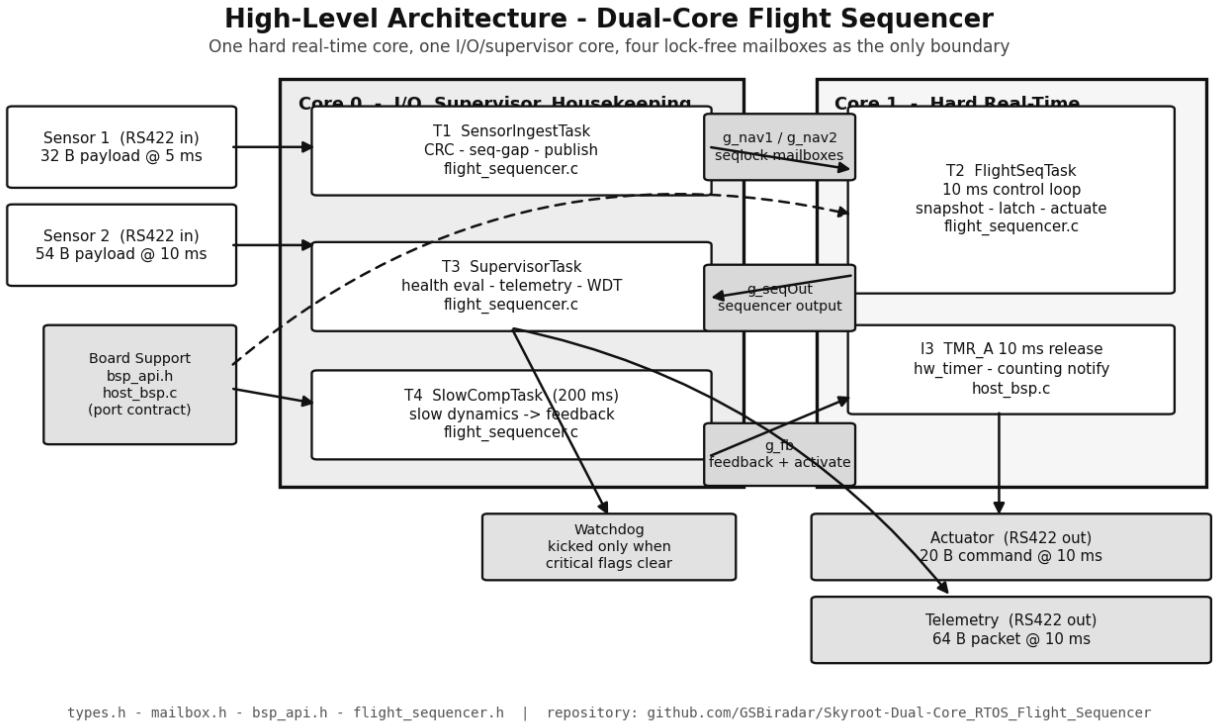


Figure 1. High-level architecture: the dual-core split, task placement, external RS422 interfaces and the four lock-free mailboxes that form the only cross-core boundary.

4.2 Low-Level Architecture

The low-level view shows the concrete implementation units and the source file that owns each one. On Core 0 a UART RX DMA ring per sensor raises one idle-line interrupt (I1/I2) which notifies the single ingest task (T1); T1 validates the frame and publishes it into a seqlock mailbox. On Core 1 a hardware timer (I3) releases the Flight Sequencer (T2), which snapshots the navigation mailboxes, latches feedback only on a 20-frame boundary, publishes g_seqOut and starts the actuator DMA. The slow task (T4) and the supervisor (T3) live on Core 0 and reach the sequencer only through the mailboxes and the shared atomic health block.

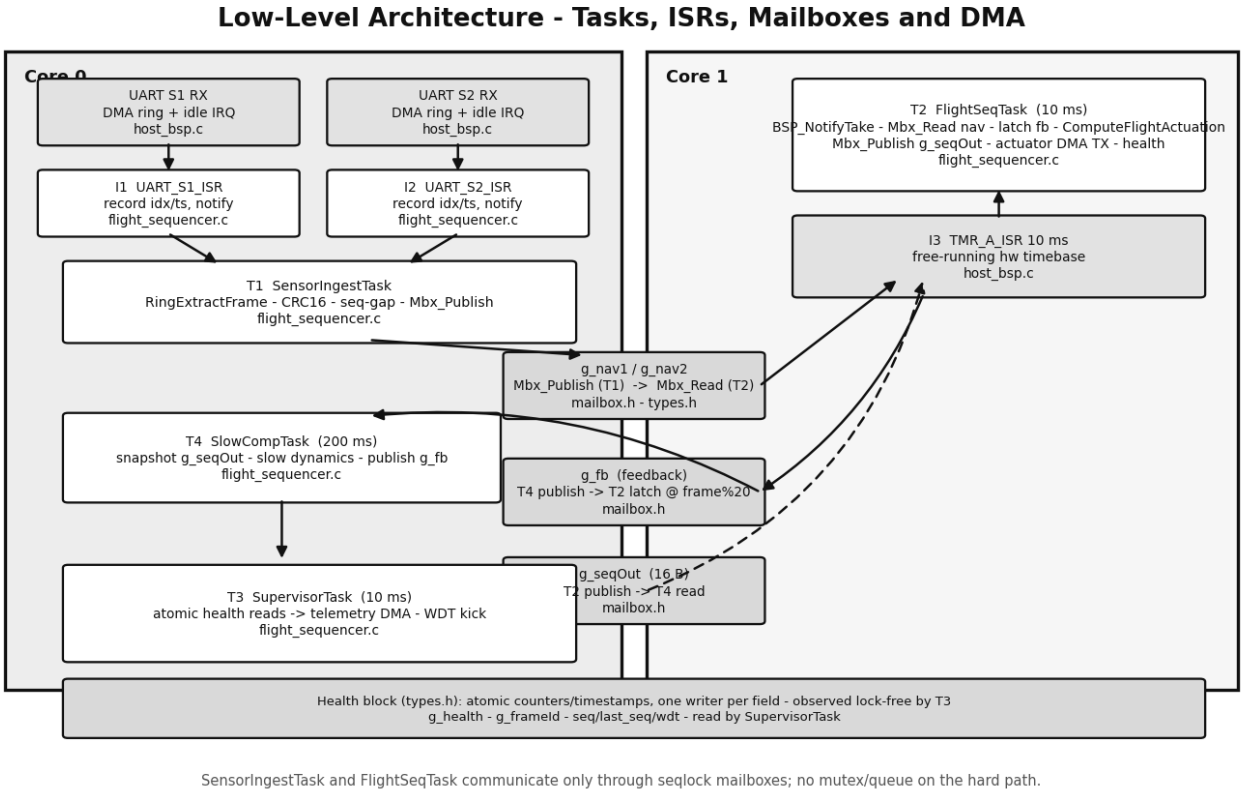


Figure 2. Low-level architecture: ISRs, DMA rings, the seqlock mailboxes (g_nav1, g_nav2, g_fb, g_seqOut) and the shared health block, annotated with the implementing source files.

Implementing files: interrupts, tasks, mailboxes and health live in src/modules/flight_sequencer.c; the shared types and sizes are in include/types.h; the seqlock protocol is header-only in include/mailbox.h; the port contract is include/bsp_api.h, implemented for the host by platform/host_bsp.c; the entry point is src/main.c; and run-time parameters are in config/simulation.h. All of these live in the mission_computer/ package of the repository.

4.3 Tasks, Timers and Interrupts

ID	Name	Kind	Core	Prio	Trigger / period	Budget
I1	UART_S1_ISR	ISR	0	IRQ 1	Sensor 1 frame end, 5 ms	5 us
I2	UART_S2_ISR	ISR	0	IRQ 1	Sensor 2 frame end, 10 ms	5 us
I3	TMR_A_ISR	ISR	1	IRQ 2	10 ms, phase 0	3 us
I4	TMR_B_ISR	ISR	0	IRQ 2	10 ms, phase +6 ms	3 us
T1	SensorIngestTask	task	0	5	notified by I1/I2	40 us/fr
T2	FlightSeqTask	task	1	5	released by I3, 10 ms	3 ms
T3	SupervisorTask	task	0	4	released by I4, 10 ms	200 us
T4	SlowCompTask	task	0	2	every 20th frame, 200 ms	40 ms

Priority ordering, higher number meaning more urgent: ISRs above all tasks; then on the application side FlightSeq (5) equals ingest (5) but on a different core; Supervisor (4) then SlowComp (2). All interrupt priorities are numerically at or below the kernel syscall ceiling, so the FromISR APIs are legal.

4.4 Data Flow and IPC Map

```
Sensor1 (5 ms)  --DMA ring--> UART_S1_ISR --notify(EVT_S1)--> T1 Ingest
Sensor2 (10 ms) --DMA ring--> UART_S2_ISR --notify(EVT_S2)--> T1 Ingest
                                                               |
                    seqlock mailbox g_nav1 / g_nav2  <---------+
                                                               |
```

```
          +-----------------+-----------------+--------------+
          v                                   v
     T2 FlightSeq (Core 1)               T4 SlowComp (Core 0)
     - snapshot nav                      - snapshot nav + g_seqOut
     - latch feedback @ frame%20         - CalculateSlowDynamics
     - ComputeFlightActuation            - publish g_fb + activate_frame
     - publish g_seqOut (seqlock)  ------>
     - actuator DMA TX (20 B)
     - health counters
          ^
          |  g_fb (seqlock, latched at fixed boundary)
          |
T3 Supervisor (10 ms) --atomic reads--> telemetry --DMA TX(UART_TLM)
      |
      +--> Wdt_Kick() only if critical flags clear
```

Data / event	Mechanism	Why
Frame received	Direct bit notification (ISR to T1)	Lightest unblock primitive, same core, no allocation.
Nav 1 / Nav 2 latest	Seqlock latest-value mailbox	Only the newest sample matters; a queue would grow stale or drop.
Periodic release	Counting notification (I3 to T2; I4 to T3/T4)	Hardware-driven release; the count also exposes missed releases.
Sequencer output (16 B)	Seqlock mailbox g_seqOut	Wait-free for the 10 ms writer.
Feedback (16 B)	Seqlock mailbox g_fb + activate_frame	Reader never blocks on the slow task; result activates at a fixed boundary.
Actuator command (20 B)	T2-owned DMA buffer	Single owner, nothing to share.
Health data	Atomic counters/timestamps, one writer per field	Supervisor observes faults with no lock.
Watchdog	Kicked only by T3 when critical flags are clear	Converts missed timing or stale data into observable reset behaviour.

4.5 Hardware Interface Strategy: Interrupts versus Polling

Per-frame costs at 1 Mbaud are 10 microseconds per byte. Sensor 1 (38 B) occupies about 0.38 ms of its 5 ms period and Sensor 2 (60 B) about 0.60 ms of its 10 ms period. A per-byte RX interrupt strategy would generate roughly 13,000 interrupts per second; polling would require a tight poll interval during each burst and would burn CPU.

Option	Pros	Cons for this timing budget
Per-byte RX interrupt	Simple, no DMA	About 13,000 ISRs/s at about 1.5 us each (about 2 percent CPU) and a preemption every 10 us during each 0.4 to 0.6 ms burst. Large, unpredictable jitter.
Polling (busy or fast task)	Fully deterministic, easiest to certify	Must poll at least every 160 us during bursts. Costs about 7 percent (S1) plus 6 percent (S2) of a core and starves lower tasks; fails if a frame is late or a poll slips.
DMA + idle-line IRQ (chosen)	One ISR per frame (about 300/s, about 0.05 percent CPU); bytes land with no CPU cost; idle-line gives automatic frame resync	Needs DMA and ring-buffer handling; ISR latency adds a small, bounded jitter.

Decision: use interrupts where arrival is asynchronous (the two sensor receives) and polled DMA completion where the flight software controls the timing (actuator and telemetry transmit). A rejected actuator frame is counted, never retried in the loop.

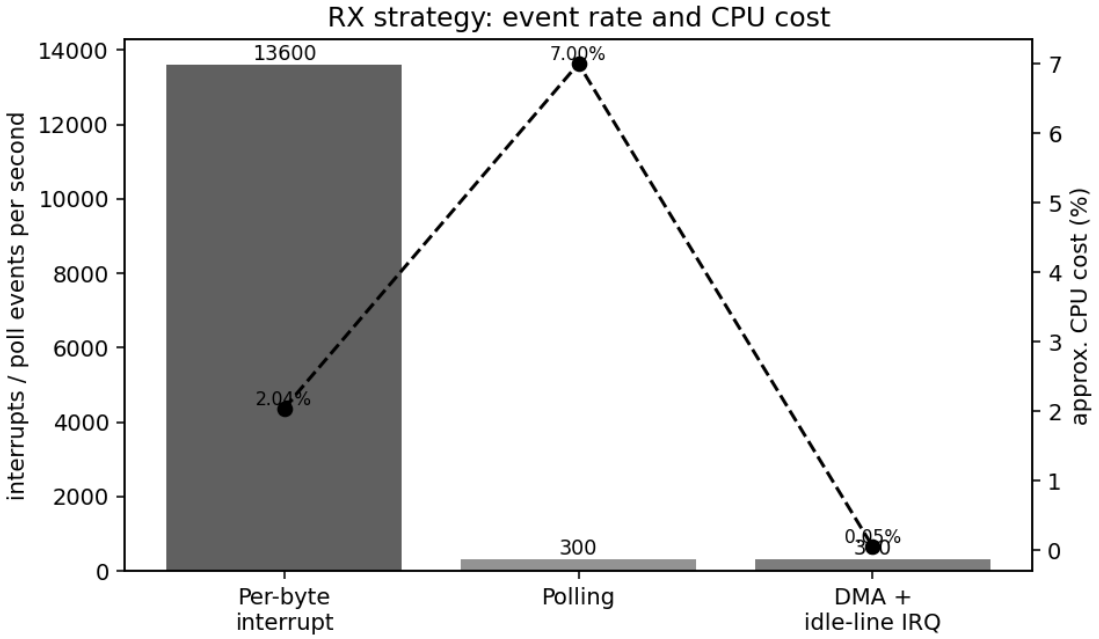


Figure 3. RX strategy: event rate and approximate CPU cost.

5. Timing and Determinism Analysis

5.1 Interrupt Load

Each sensor needs one interrupt per frame, so both sensors together produce about 300 interrupts per second at roughly 5 microseconds each -- about 0.05 percent of a core. This is why DMA plus idle-line interrupt beats per-byte interrupts by two orders of magnitude.

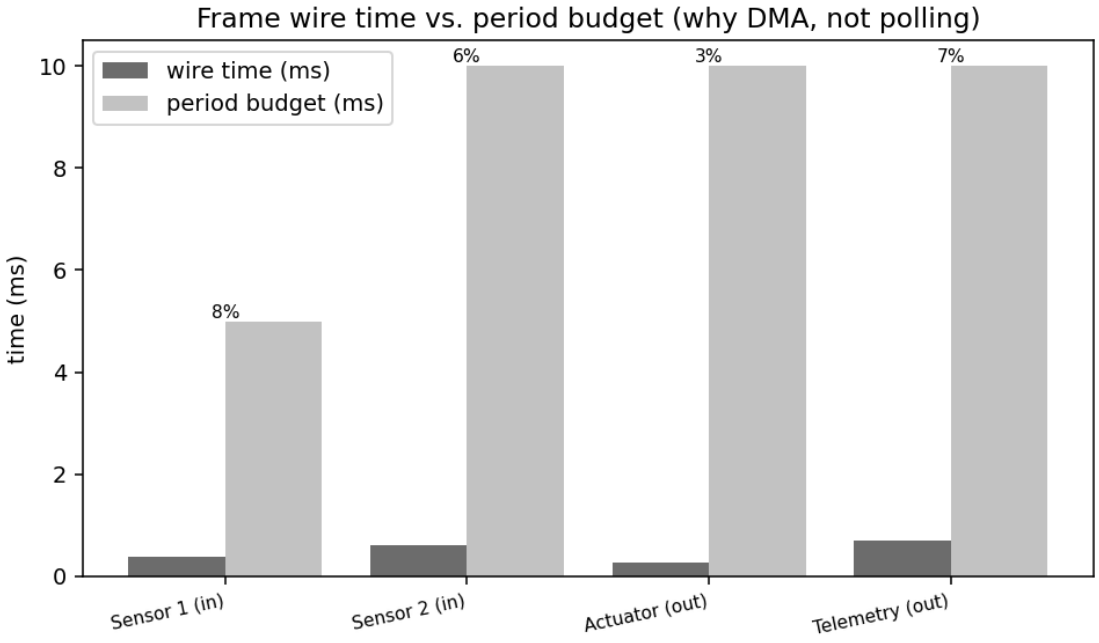


Figure 4. Frame wire time versus period budget: why DMA is required.

5.2 Worst-Case Response Times

The Flight Sequencer owns Core 1, so its worst-case response is its own WCET plus the timer ISR plus a bus-contention allowance: about 3 ms + 3 us + 30 percent, approximately 3.9 ms against a 10 ms deadline. That leaves about 6 ms of slack. Sensor ingest on Core 0 has a worst case of ISR (5 us) + at most one other

ingest in progress (40 us) + its own work (40 us), under 100 us, about 2 percent of the 5 ms period.

Activity	Worst-case response	Deadline	Slack
Sensor ingest	about 0.1 ms	5 ms	about 4.9 ms
Flight Sequencer	about 3.9 ms	10 ms	about 6.1 ms
Supervisor	about 0.2 ms	10 ms	about 9.8 ms
Slow computation	about 45 ms	200 ms	about 155 ms

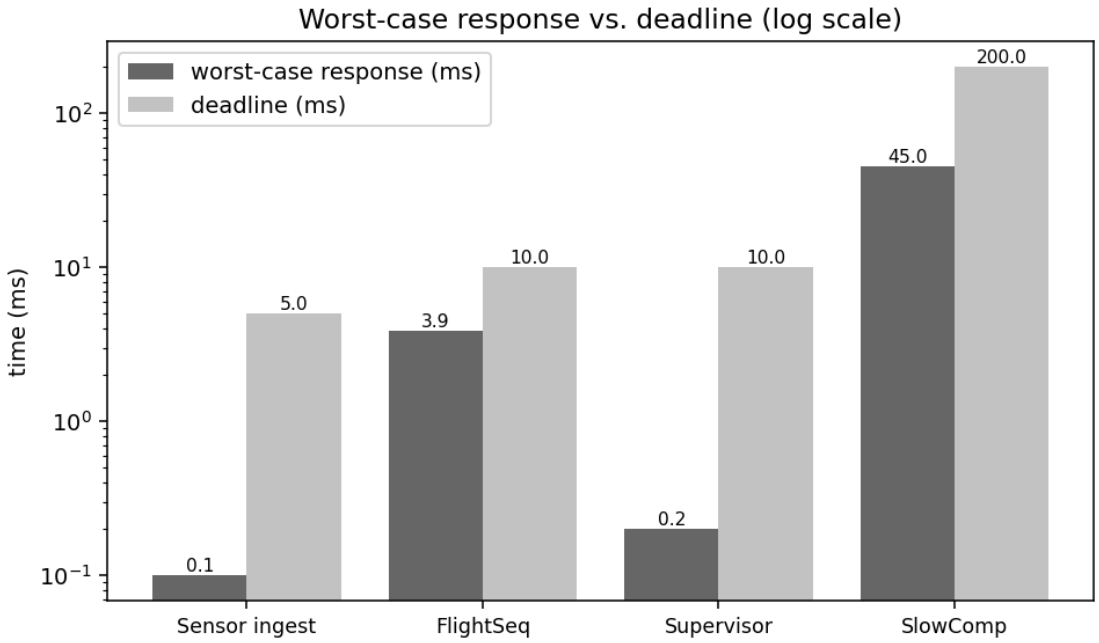


Figure 5. Worst-case response versus deadline (log scale).

5.3 CPU Utilisation

Core 0 utilisation: ISRs and ingest about 1.1 percent, supervisor about 1.5 percent, slow task about 20 percent, total about 23 percent. Core 1 utilisation: the 3 ms sequencer in a 10 ms period is about 30 percent. Core 1 is isolated from all asynchronous load, which is the key determinism decision.

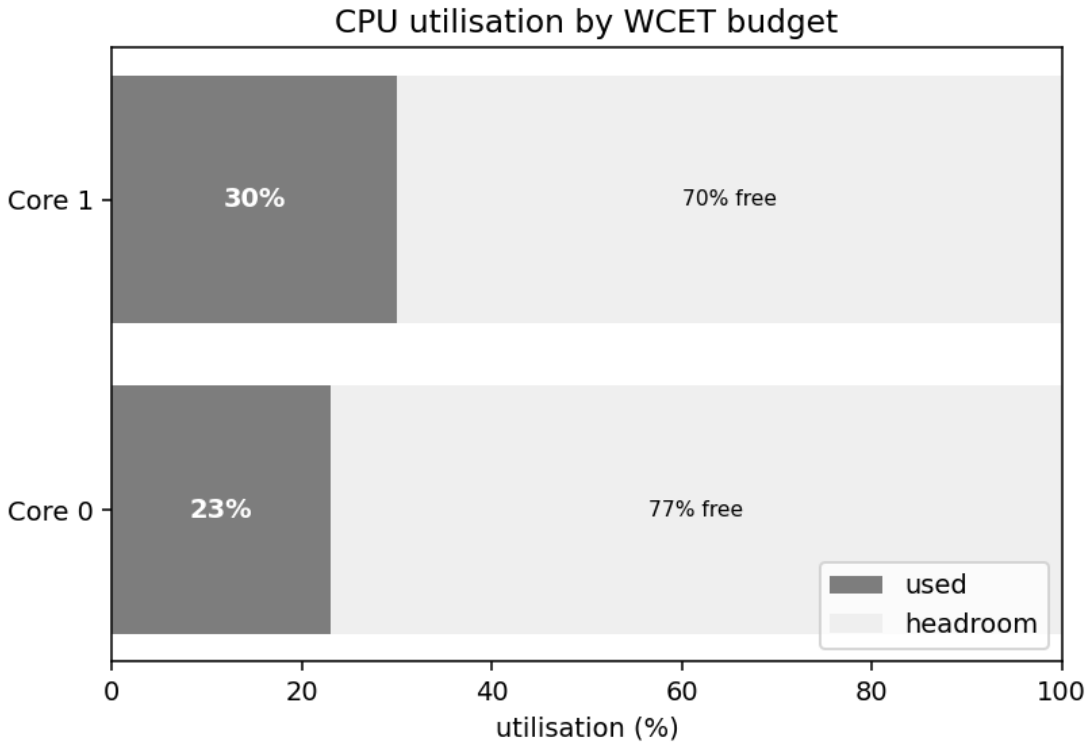


Figure 6. CPU utilisation by WCET budget.

5.4 Data Age and Staleness

With unsynchronised sensors, the worst-case data age at the sequencer is under 5.5 ms for Sensor 1 and under 10.7 ms for Sensor 2. The staleness limits are 7.5 ms and 15 ms (1.5 periods); exceeding a limit means hold the last good value and raise a flag. Worst-case sample-to-actuator latency is about 9.6 ms for Sensor 1 and about 14.9 ms for Sensor 2.

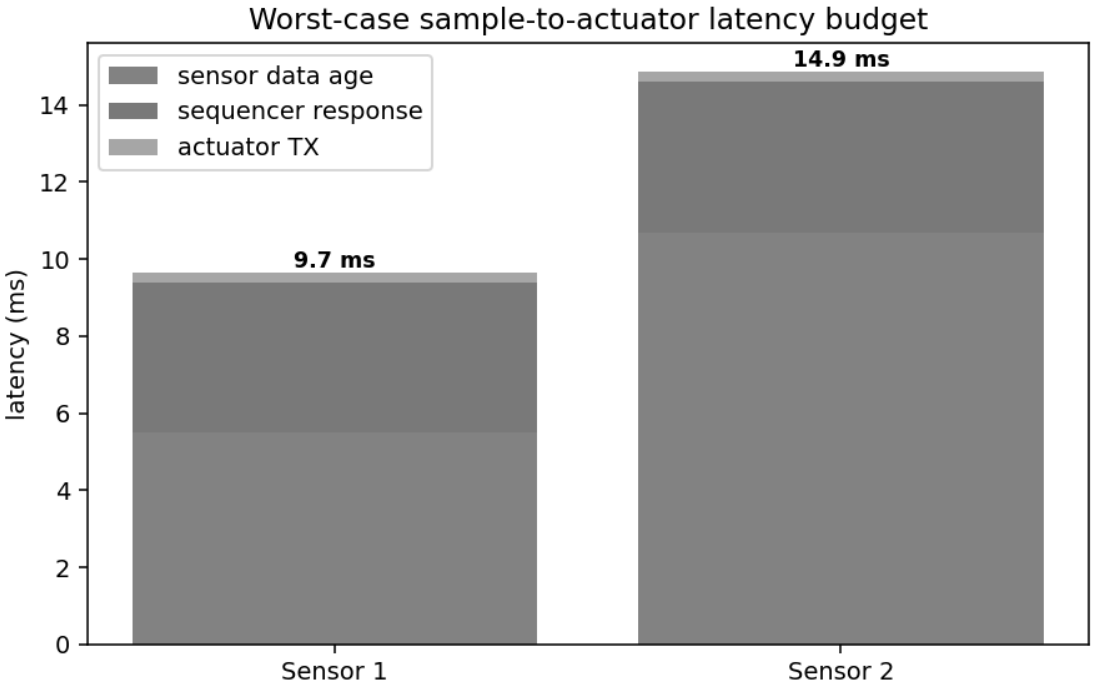


Figure 7. Worst-case sample-to-actuator latency budget.

5.5 Overrun and Missed-Release Detection

Counting notifications reveal missed releases, and any WCET breach is reported in telemetry and gates the watchdog. This turns any future violation into an observable signal rather than silent drift.

6. Synchronization and Concurrency

6.1 Shared Resources

Resource	Writer	Readers	Critical section
g_nav1 (32 B + ts)	T1 (Core 0)	T2 (Core 1), T4	Mailbox write / copy
g_nav2 (54 B + ts)	T1 (Core 0)	T2 (Core 1), T4	Mailbox write / copy
g_seqOut (16 B)	T2 (Core 1)	T4 (Core 0)	Mailbox write / copy
g_fb (16 B feedback)	T4 (Core 0)	T2 (Core 1)	Mailbox write / copy
Health block	One writer per field	T3	None (atomic word access)
UART RX rings	DMA	I1/I2 then T1	Producer-index handoff via ISR
TX buffers	T2 / T3	DMA	Owned by sender; busy flag checked first
g_frameId	I3	T2, I4	Single atomic word

6.2 The Seqlock Mailbox Protocol

A seqlock carries a sequence counter: even means the snapshot is stable, odd means a write is in progress. The writer is wait-free; the reader copies, rechecks the counter, and retries a bounded number of times. The writer takes a roughly 0.2 microsecond critical section with local-core interrupts masked, so it cannot be preempted mid-write on its own core.

```
static inline void Mbx_Publish(Mbx_t *m, const void *src, size_t n,
                               uint64_t ts_us, uint32_t frame_id)
{
    uint32_t s = atomic_load_explicit(&m->seq, memory_order_relaxed);
    atomic_store_explicit(&m->seq, s + 1u, memory_order_relaxed); /* odd */
    atomic_thread_fence(memory_order_release);
    memcpy(m->data, src, n);  m->ts_us = ts_us;  m->frame_id = frame_id;
    atomic_store_explicit(&m->seq, s + 2u, memory_order_release); /* even */
}

static inline bool Mbx_Read(const Mbx_t *m, void *dst, size_t n,
                            uint64_t *ts_us, uint32_t *frame_id)
{
    for (uint32_t i = 0u; i < MBX_MAX_RETRY; i++) {
        uint32_t s1 = atomic_load_explicit(&m->seq, memory_order_acquire);
        if ((s1 & 1u) != 0u) continue;              /* writer active */
        memcpy(dst, m->data, n);
        atomic_thread_fence(memory_order_acquire);
        if (s1 == atomic_load_explicit(&m->seq, memory_order_relaxed))
            return true;                            /* stable snapshot */
    }
    return false;                                   /* keep last good */
}
```

6.3 Correctness Arguments

- No torn reads: the reader accepts a snapshot only when the sequence counter is unchanged across the copy, with release/acquire ordering (DMB on ARM).
- No race on latest-value reads: readers take the freshest complete value and never block; staleness is detected by a wrap-safe timestamp and per-source sequence.
- No deadlock: no nested locks, no blocking call while holding exclusion, and no lock shared between cores. The only exclusion is a local IRQ mask of bounded length.

- No priority inversion: the high-priority consumer never waits on a lower-priority producer. Cross-core priority inheritance does not exist, which is a further reason to avoid mutexes on the hard path; any mutex is confined to non-critical Core 0 code.
- Different task frequencies: latest-value semantics decouple the 5, 10 and 200 ms rates; each consumer snapshots at the start of its cycle and works on the copy.
- Feedback loop integrity: g_fb carries activate_frame, so T2 latches new feedback only on a 20-frame boundary; output is deterministic and the previous value is held (with a stale counter) if the slow task is late.

7. Implementation

7.1 File Layout

Path	Responsibility
include/types.h	Sizes, rates, frame format, Health_t, Mbx_t, payload types
include/mailbox.h	Header-only seqlock publish/read
include/bsp_api.h	The only interface the application depends on
src/modules/flight_sequencer.c	Tasks T1-T4, ISRs I1-I4, init and report
src/main.c	Host entry: init, run, report
platform/host_bsp.c	POSIX host port + fault-injecting hardware simulator
config/simulation.h	Run length and injected fault rates
Makefile / CMakeLists.txt	Host builds
scripts/generate_timing_plots.py	Regenerate the timing figures
scripts/generate_report.py	Build this report
docs/*.md	Architecture, decisions, validation, traceability, integration

The complete package is published at github.com/GSBiradar/Skyroot-Dual-Core_RTOS_Flight_Sequencer under the mission_computer/ directory. Every source file, header, build file, figure-generation script and document listed above is committed there; the two architecture diagrams and the timing figures in this report are reproduced by the scripts in scripts/.

7.2 The Board-Support Contract

The application is written only against include/bsp_api.h. On the target that contract is implemented over FreeRTOS-SMP or CMSIS-RTOS2; on a PC it is implemented by platform/host_bsp.c. The contract covers time, task creation with core affinity, bit and counting notifications, a local IRQ mask, hardware timers, RS422/UART with DMA, and the watchdog. Because the contract is small, porting is a bounded task.

7.3 Sensor Ingest (T1)

The UART idle-line ISR records the DMA write index and timestamp, then notifies the ingest task. The ingest task pulls the newest complete frame out of the circular ring, validates the CRC, tracks sequence gaps, and publishes the payload to the navigation mailbox. Bad frames are discarded and counted; the last good snapshot is retained.

```
static void IngestOne(RxCtx_t *c, Mbx_t *mbx, uint32_t uart, uint32_t len, ...)
{
    Frame_t f;
    if (!RingExtractFrame(uart, len, &f)) { atomic_fetch_add(sync_err, 1); return; }
    uint16_t stored = f.raw[f.n-2] | (f.raw[f.n-1] << 8);
    if (Crc16(f.raw, f.n - 2u) != stored) { atomic_fetch_add(crc_err, 1); return; }
    uint8_t gap = (uint8_t)(f.seq - c->last_seq - 1u);
    if (gap) atomic_fetch_add(seq_gap, gap);
    c->last_seq = f.seq;
```

```
    Mbx_Publish(mbx, f.payload, len, c->isr_ts, 0u);
    atomic_store(last_ts, (uint32_t)c->isr_ts);
    atomic_fetch_add(frames, 1);
}
```

7.4 Flight Sequencer (T2)

The sequencer waits on a counting notification from the 10 ms timer. It reads the latest navigation snapshots (keeping the last good copy on a failed read), checks freshness, latches feedback only on a 20-frame boundary, computes the actuation, publishes the 16-byte output, starts the 20-byte actuator DMA transmit, and updates its execution-time and heartbeat counters. It never blocks on data and never busy-waits on transmit.

```
uint32_t rel = BSP_NotifyTake(true, BSP_WAIT_FOREVER);
if (rel > 1u) atomic_fetch_add(&g_health.seq_missed_rel, rel - 1u);
uint64_t t0 = BSP_HwTimeUs();
uint32_t frame = atomic_load(&g_frameId);

if (Mbx_Read(&g_nav1, &t1, N1_LEN, &ts1, &d)) n1 = t1;   /* keep last good */
if (Mbx_Read(&g_nav2, &t2, N2_LEN, &ts2, &d)) n2 = t2;
bool v1 = ((t0 - ts1) <= NAV1_MAX_AGE_US);
bool v2 = ((t0 - ts2) <= NAV2_MAX_AGE_US);

if ((frame % SLOW_DIV) == 0u) {                          /* fixed boundary */
    if (Mbx_Read(&g_fb, &fbTmp, sizeof(fbTmp), &tsf, &d) &&
        fbTmp.cycle_id != fbCycle && fbTmp.activate_frame <= frame)
    { fb = fbTmp.payload; fbCycle = fbTmp.cycle_id; }
    else atomic_fetch_add(&g_health.fb_stale_cnt, 1u);   /* hold previous */
}
ComputeFlightActuation(&st, &n1, &n2, v1, v2, &fb, &cmd, &out16);
Mbx_Publish(&g_seqOut, &out16, SEQOUT_LEN, t0, frame);
if (!BSP_DmaTxBusy(UART_ACT)) { start actuator DMA; }    /* non-blocking */
else atomic_fetch_add(&g_health.act_busy_cnt, 1u);
```

7.5 Slow Computation (T4)

The slow task is released every 20th frame by the supervisor timer. It snapshots navigation and the sequencer output, computes the slow dynamics, and publishes feedback together with the next activation boundary so the sequencer can apply it deterministically.

7.6 Supervisor and Watchdog (T3)

The supervisor runs at frame start plus 6 ms. It reads the atomic health block without locks, evaluates sensor freshness, CRC faults, sequencer heartbeat, the execution-time budget, missed releases, actuator transmit and slow-task health, builds a telemetry packet and transmits it by DMA. It kicks the watchdog only when every critical flag is clear. Fault detection is edge-triggered (it compares against the previous cycle), so a single transient event does not latch a fault forever and the watchdog keeps guarding while the system is healthy.

```
uint32_t s1c = atomic_load(&g_health.s1_crc_err);
uint32_t s2c = atomic_load(&g_health.s2_crc_err);
if (s1c > lastS1Crc || s2c > lastS2Crc) flags |= F_NAV_CRC;   /* new faults only */
lastS1Crc = s1c;  lastS2Crc = s2c;
if (atomic_load(&g_health.seq_exec_us) > SEQ_BUDGET_US) flags |= F_SEQ_OVERRUN;
...
if ((flags & F_CRITICAL_MASK) == 0u) { BSP_WdtKick(); }        /* gate */
```

8. Building and Running

8.1 Windows (MinGW-w64 gcc)

```
gcc -std=c11 -O2 -Iinclude -Iconfig \
    src/main.c src/modules/flight_sequencer.c platform/host_bsp.c \
    -o flight_seq.exe -pthread
.\flight_seq.exe            # runs about 3 s, prints a report, exits
```

8.2 Linux / WSL

```
sudo apt install -y build-essential
make run                    # build + run
make run ARGS=10            # 10 second soak
```

8.3 Make and CMake

```
make            # build flight_seq(.exe)
make run        # build then run
make clean      # remove build artefacts

cmake -S . -B build && cmake --build build
./build/flight_seq 3
```

The program takes one optional argument: the run length in seconds (default 3, from config/simulation.h).

9. Validation

9.1 Test Plan

#	Test	Pass criterion
V1	Periodic release	FlightSeq runs every 10 ms; missed_rel == 0
V2	Sensor ingest + framing + CRC	crc > 0; counts consistent; bad frames discarded
V3	Staleness / hold-last-good	stale or missing input holds last good; safe fallback
V4	Seqlock integrity	no torn snapshots; timeouts == 0 on target
V5	Feedback fixed boundary	new feedback applied only at frame % 20 == 0
V6	Watchdog gating	kicks high, skips track fault windows
V7	Non-blocking TX	busy/reject counted; the run always completes
V8	WCET	seq_exec_us stays well below the 3000 us budget

9.2 Observed Results

A representative three second host run:

```
============== Flight Sequencer health report (dual-core) ==============
 sensor1  frames / crc / seq_gap / sync_err : 593 / 7 / 7 / 7
 sensor2  frames / crc / seq_gap / sync_err : 295 / 5 / 5 / 3
 flight   heartbeats / missed_rel / max_us  : 301 / 0 / 4
 actuator tx / tx_err / busy_cnt           : 292 / 2 / 7
 slow     heartbeats / exec_us / fb_stale   : 15 / 0 / 1
 supervisor heartbeats / tlm / tlm_busy    : 300 / 299 / 1
 watchdog kicks / skips                     : 287 / 13
 ---------------- seqlock + timing invariants ----------------
 seqlock reads / retries / timeouts         : 663 / 0 / 0
 last supervisor flag word                  : 0x00000000
```

Interpretation: frame counts match the periods (about 600 at 5 ms, about 300 at 10 ms); the crc and seq_gap counts match at about 1 percent injected corruption because a discarded frame shows up as a sequence hole; the sequencer ran every release with zero missed releases and a maximum execution well under budget; actuator and telemetry transmits were non-blocking (busy counted, never a hang); the slow loop produced about 15 feedback cycles; the watchdog was kicked 287 times with 13 transient skips corresponding to fault windows; the seqlock recorded zero torn snapshots; and the final flag word is zero, meaning the system ended healthy. Numeric values vary between runs because faults are injected at random.

10. Improvements over the Base Pseudocode

The assessment pseudocode is a strong starting point. The reference implementation keeps its architecture and corrects a few details.

#	Issue in base pseudocode	Fix in this implementation
I1	Supervisor flags F_NAV_CRC from a cumulative counter, so one CRC error latches the fault forever and the watchdog is never kicked again.	Edge-triggered fault detection: compare against the previous cycle and flag only new faults; keep the watchdog guarding while healthy.
I2	Reads FB_LEN (16) bytes into a structure that also holds cycle_id and activate_frame.	Publishes/reads sizeof(FbMsg_t) so the control fields travel atomically with the 16-byte payload.
I3	seq - last_seq - 1 with last_seq = 0 reports a bogus 255-frame gap on the first frame.	Pre-load last_seq to 0xFF so the first frame is gap-free.
I4	Simulator corrupts a frame then recomputes a matching CRC, so the CRC check never fails.	Seal the CRC over the clean frame, then flip a payload byte.
I5	act_busy / tlm_busy treated as latched faults.	Reported as counters and used edge-triggered, matching real FDIR practice.

11. Evaluation Traceability

Criterion	How the design meets it	Where
Real-time determinism	Isolated hard real-time core, timer release, DMA RX, wait-free writers, bounded readers, response-time analysis with about 6 ms slack, missed-release and overrun detection.	Sections 4, 5; flight_sequencer.c
RTOS fundamentals	Notifications for events, counting notifications for periodic release, seqlock mailboxes for state, single-owner DMA for TX, no mutexes on the hard path.	Sections 4.3, 6; bsp_api.h
Multi-core competency	Core 1 dedicated to the sequencer; four single-writer mailboxes as the only boundary; memory barriers; non-cacheable shared RAM; false-sharing control; local IRQ mask instead of the SMP kernel lock.	Sections 4.1, 6; types.h, mailbox.h
Engineering judgment	Nine documented assumptions with WCET budgets and frame formats; stale-data policy; fixed-boundary feedback activation.	Section 3.3; DESIGN_DECISIONS.md
Design justification	Quantified interrupt-vs-polling comparison and lock-free-vs-mutex rationale.	Sections 4.4, 6.3

12. Porting and Integration

Only platform/host_bsp.c is host-specific. To move to a real target, re-implement include/bsp_api.h over your RTOS and vendor drivers; the application does not change.

BSP function	FreeRTOS-SMP	CMSIS-RTOS2
BSP_TaskCreate	xTaskCreateAffinitySet	osThreadNew + affinity
BSP_NotifySetBitsFromIsr	xTaskNotifyFromISR (eSetBits)	osThreadFlagsSet
BSP_NotifyWaitBits	xTaskNotifyWait	osThreadFlagsWait
BSP_NotifyGiveFromIsr	vTaskNotifyGiveFromISR	counting semaphore
BSP_NotifyTake	ulTaskNotifyTake	osSemaphoreAcquire
BSP_LocalIrqSave/Restore	portSET_INTERRUPT_MASK (local core)	__disable_irq region
BSP_DmaRx/Tx	vendor UART DMA	vendor UART DMA
BSP_WdtKick	wdt_feed	HAL_IWDG_Refresh

Shared memory and memory ordering are critical: map shared RAM non-cacheable (or coherent), 64-byte align every mailbox and health block, and confirm the acquire/release fences map to DMB on ARM. Route UART interrupts to Core 0 and timer A to Core 1. Do not enable the watchdog until the supervisor is proven to kick it.

13. Conclusion

The Flight Sequencer demonstrates a defensible, certifiable hard real-time architecture: a dedicated compute core, hardware-driven release, DMA-based I/O, wait-free state exchange and lock-free fault observation, all with quantified margins and explicit failure behaviour. The accompanying implementation is not a diagram -- it builds and runs on Windows and Linux, exercises its safety paths under injected faults, and self-reports its health so the timing and synchronization claims can be observed directly.

Appendix A. Constants and Frame Format

Constant	Value	Meaning
FRAME_US	10000	Nominal 10 ms flight frame
SLOW_DIV	20	Slow task every 20th frame (200 ms)
SENSOR1_PERIOD_US	5000	Sensor 1 period
SENSOR2_PERIOD_US	10000	Sensor 2 period
N1_LEN / N2_LEN	32 / 54	Sensor payload lengths (bytes)
ACT_LEN / FB_LEN	20 / 16	Actuator command / feedback lengths
FRAME_OVERHEAD	6	2 sync + 1 seq + 1 len + 2 crc
BAUD_RATE	1000000	1 Mbaud, 8N1 (10 us/byte)
NAV1_MAX_AGE_US	7500	1.5 periods staleness limit
NAV2_MAX_AGE_US	15000	1.5 periods staleness limit
SEQ_BUDGET_US	3000	Flight sequencer WCET budget
MBX_MAX_RETRY	3	Bounded seqlock reader retries

```
Frame:  [0xAA][0x55][seq][len][ payload ... ][crc_lo][crc_hi]
CRC-16/CCITT-FALSE over every byte except the two CRC bytes.
```

Appendix B. Board-Support API Reference

Function	Purpose
BSP_ClockInit / BSP_HwTimeUs/Ms	Initialise and read the shared 1 us timebase
BSP_TaskCreate	Create a statically allocated task pinned to a core
BSP_NotifySetBitsFromIsr / BSP_NotifyWaitBits	Bit notification for events
BSP_NotifyGiveFromIsr / BSP_NotifyTake	Counting notification for periodic release
BSP_LocalIrqSave / BSP_LocalIrqRestore	Bounded per-core interrupt mask
BSP_TimerSetIsr / StartPeriodic / ClearIrq	Hardware timer release
BSP_Rs422Init / UartSetIdleIsr / UartEnableIdleIrq	UART + idle-line IRQ setup
BSP_UartStatus / ClearIdle / ClearErr	Status and flag handling
BSP_DmaRxCircular / RingBuffer / RingSize / DmaWriteIndex	Circular RX DMA
BSP_DmaTxStart / DmaTxBusy	One-shot TX DMA with polled completion
BSP_WdtKick	Watchdog feed (supervisor only)
BSP_RunForSeconds	Host-only: run the simulator for N seconds

Appendix C. Repository Layout

```
mission_computer/
  CMakeLists.txt, Makefile, README.md
  config/simulation.h
  include/  types.h  mailbox.h  bsp_api.h  flight_sequencer.h
  platform/ host_bsp.c
  src/      main.c  modules/flight_sequencer.c
  scripts/  generate_architecture_diagrams.py
            generate_timing_plots.py  generate_report.py
  docs/     ARCHITECTURE.md  DESIGN_DECISIONS.md  VALIDATION.md
            TRACEABILITY.md  INTEGRATION.md  figures/  figures_bw/
            Flight_Sequencer_Report.pdf
```

Appendix D. Glossary

Term	Definition
Seqlock	Sequence-lock; a lock-free data exchange where the writer is wait-free and the reader retries on overlap.
Wait-free	An operation guaranteed to complete in a bounded number of steps regardless of other threads.
Counting notification	A task signal that accumulates a count, so missed releases can be detected.
Edge-triggered fault	A fault flagged only at the transition (new event), rather than for as long as a cumulative counter is non-zero.
WCET	Worst-Case Execution Time.
False sharing	Performance loss when independent variables share a cache line; avoided by 64-byte alignment.